

Lecture 07: Introduction to Arrays

Is lecture mein hum **Arrays** ko bilkul shuru se samajhenge — kya hai, kyun zaroori hai, memory mein kaise store hote hain, aur kaise use karte hain. Array programming ka sabse important data structure hai. Agar aap first time arrays padh rahe ho, toh ye notes aapke liye perfect hai.



The Problem — Why We Need Arrays

Socho aapko **5 students ke marks** store karne hain. Bina array ke aapko 5 alag-alag variables banane padenge:

```
int marks1 = 70;
int marks2 = 40;
int marks3 = 80;
int marks4 = 14;
int marks5 = 18;

// Print karna ho toh 5 baar cout
cout << marks1 << endl;
cout << marks2 << endl;
// ... aur aage 3 aur lines
```

IMPORTANT

Ab socho **100 students** ke marks store karne hain! Bina array ke **100 variables** banane padenge — marks1, marks2, marks3... marks100. Ye impossible hai. Isliye **Array** aata hai bachane.

Key Insight: Array ek aisa container hai jisme **ek hi type ke multiple values** ek saath store ho sakti hain — aur unhe ek **index number** se access kiya ja sakta hai.



What is an Array?

Array ek **collection of similar data types** hai jo **continuous memory locations** mein store hote hain. Har element ko ek **index number** se access kiya ja sakta hai.

INTUITION

Array ko ek **building ki floors** ki tarah samjho. Har floor pe ek family rehti hai (value). Ground floor = 0, First floor = 1, Second floor = 2... Har floor ka ek **unique number** hai jisse aap directly us floor pe pahunch sakte ho. Array mein ye number **index** hota hai.

Array Declaration

```
int marks[5]; // 5 integers ka array declare kiya
```

Is line ka matlab:

- int → Array mein integers store honge
- marks → Array ka naam
- [5] → Array mein 5 elements aa sakte hain

Visual Representation

◆ ARRAY VISUAL: marks[5]

Index 0	Index 1	Index 2	Index 3	Index 4
70	40	80	14	18
0x1000	0x1004	0x1008	0x100C	0x1010

✅ Continuous memory — har element 4 bytes door



Array Initialization

METHOD 1: INDEX BY INDEX

```
int marks[5];    // Array declare

marks[0] = 70;   // Index 0 pe 70 daala
marks[1] = 40;   // Index 1 pe 40 daala
marks[2] = 80;   // Index 2 pe 80 daala
marks[3] = 14;   // Index 3 pe 14 daala
marks[4] = 18;   // Index 4 pe 18 daala
```

METHOD 2: DIRECT INITIALIZATION

```
int marks[5] = {70, 40, 80, 14, 18};

// Ya phir size bhi chhod sakte ho
int marks[] = {70, 40, 80, 14, 18}; // Compiler khud size 5 le lega
```

WORKFLOW

- **Step 1:** Array declare karo — `int marks[5];`
- **Step 2:** Values daalo — `marks[0] = 70;`
- **Step 3:** Values access karo — `cout << marks[0];`



0-Based Indexing — Very Important

IMPORTANT

C++ mein array **hamesha 0 se start hota hai**. Agar aapne `int arr[5]` declare kiya, toh valid indexes hain: **0, 1, 2, 3, 4**. `arr[5]` access karna **galat hai** — ye array ke bahar hai.

◆ 0-BASED INDEXING

0	1	2	3	4
70	40	80	14	18

⚠ `arr[5] = ❌ OUT OF BOUNDS! (Index 5 exist nahi karta)`

Key Insight: Agar array ka size n hai, toh last valid index $n-1$ hota hai. `int arr[5]` ke liye last index 4 hai, 5 nahi. Ye sabse common beginner mistake hai.



Traversing an Array (Loop se)

Array ke sabhi elements ko ek-ek karke access karna **traversing** kehlata hai. Iske liye for loop sabse best hai.

```
#include <iostream>
using namespace std;

int main() {
    int marks[5] = {70, 40, 80, 14, 18};

    // Array ko traverse kar rahe hain
    for(int i = 0; i < 5; i++) {
        cout << "marks[" << i << "] = " << marks[i] << endl;
    }

    return 0;
}
```

WORKFLOW — LOOP EXECUTION			
Iteration	i	marks[i]	Output
1	0	marks[0]	70
2	1	marks[1]	40
3	2	marks[2]	80
4	3	marks[3]	14
5	4	marks[4]	18



Array Input and Output

USER SE INPUT LENA

```

#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Tell me the size of an Array: ";
    cin >> n;

    int arr[n]; // User ke bataye size ka array

    cout << "Enter the values in an Array:" << endl;

    // Input loop
    for(int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // Output loop
    cout << "Array elements are:" << endl;
    for(int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}

```

EDGE CASE

Warning: Kuch compilers mein `int arr[n];` (variable size) allow nahi hota. Isliye **constant size** use karo — `int arr[100];` aur phir actual `n` elements hi input lo. Ya phir **dynamic allocation** use karo (baad mein padhenge).



Memory Layout — How Array is Stored

Array **continuous memory locations** mein store hota hai. Har element ka address pichle element ke address se **data type ke size** jitna door hota hai.

INT ARRAY (4 BYTES EACH)

◆ **MEMORY LAYOUT: int arr[5] — Each int = 4 bytes**

Index	0	1	2	3	4
Address	0×1000	0×1004	0×1008	0×100C	0×1010
Value	70	40	80	14	18
Size	Total = 5 × 4 = 20 bytes				

ADDRESS CALCULATION FORMULA

INTUITION

Kisi bhi element ka address nikalne ka formula:

Address of arr[i] = Base Address + (i × Size of Data Type)

Jaise arr[3] ka address: $1000 + (3 \times 4) = 1000 + 12 = 1012$ (0x100C)

DOUBLE ARRAY (8 BYTES EACH)

◆ **MEMORY LAYOUT: double salary[5] — Each double = 8 bytes**

Index	0	1	2	3	4
Address	0×2000	0×2008	0×2010	0×2018	0×2020
Value	50000.0	60000.0	45000.0	70000.0	55000.0
Size	Total = 5 × 8 = 40 bytes				

Har element 8 bytes door hai — kyunki double 8 bytes ka hota hai



1-Based Indexing Problem

Kuch languages (jaise MATLAB, R) mein array **1 se start** hota hai. Lekin C++ mein **hamesha 0 se start** hota hai. Is difference ko samajhna zaroori hai.

✅ C++ (0-Based)

Index	0	1	2	3	4
Value	70	40	80	14	18

arr[0] = 70 (First element)

❌ 1-Based (Other Languages)

Index	1	2	3	4	5
Value	70	40	80	14	18

arr[1] = 70 (First element)

IMPORTANT

C++ mein **hamesha 0 se start** karo. `for(int i = 0; i < n; i++)` — ye pattern yaad kar lo. `i < n` isliye kyunki last index `n-1` hota hai.



Memory Address Print Karna

Aap `&` operator se kisi bhi variable ya array element ka **memory address** print kar sakte ho.

```
#include <iostream>
using namespace std;

int main() {
    int a = 10;
    cout << "Address of a: " << &a << endl;

    int arr[5] = {70, 40, 80, 14, 18};

    // Har element ka address print karo
    for(int i = 0; i < 5; i++) {
        cout << "Address of arr[" << i << "]: " << &arr[i] << endl;
    }

    return 0;
}
```

WORKFLOW — EXPECTED OUTPUT

Output	Difference
Address of arr[0]: 0x1000	—
Address of arr[1]: 0x1004	+4 bytes
Address of arr[2]: 0x1008	+4 bytes
Address of arr[3]: 0x100C	+4 bytes
Address of arr[4]: 0x1010	+4 bytes



Problem 1: Find Highest Element

INTUITION

Sabse pehle maan lo **pehla element sabse bada hai** . Phir baaki sab elements se compare karo. Agar koi element current highest se bada hai, toh usse new highest bana do.

```
#include <iostream>
using namespace std;

int main() {
    int marks[5] = {70, 80, 40, 90, 20};

    // Step 1: Maan lo pehla element highest hai
    int highest = marks[0];

    // Step 2: Baaki elements se compare karo
    for(int i = 1; i < 5; i++) {
        if(marks[i] > highest) {
            highest = marks[i]; // New highest mil gaya
        }
    }

    cout << "Highest marks: " << highest << endl;

    return 0;
}
```


WORKFLOW — STEP BY STEP

Iteration	i	marks[i]	highest	Comparison
Initial	—	—	70	—
1	1	80	80	80 > 70 ✓ Update
2	2	40	80	40 > 80 ✗ No change
3	3	90	90	90 > 80 ✓ Update
4	4	20	90	20 > 90 ✗ No change



Problem 2: Search in an Array (Linear Search)

INTUITION

Array ke har element ko ek-ek karke check karo. Agar target element mil jaye, toh "Found" print karo aur loop se bahar niklo. Agar pura array check karne ke baad bhi na mile, toh "Not Found".

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {70, 80, 40, 90, 20};
    int search = 90;

    for(int i = 0; i < 5; i++) {
        if(arr[i] == search) {
            cout << "YES, Element found at index " << i << endl;
            return 0; // Program yahin khatam
        }
    }

    // Agar loop complete ho gaya aur element nahi mila
    cout << "NO, Element not found" << endl;

    return 0;
}
```

WORKFLOW — SEARCH FOR 90

Iteration	i	arr[i]	arr[i] == 90?	Action
1	0	70	No	Continue
2	1	80	No	Continue
3	2	40	No	Continue
4	3	90	Yes ✓	Found! return 0

BEST PRACTICE

Element milne ke baad `return 0;` se program end karo — isse unnecessary iterations bachte hain. Agar return nahi karte, toh loop pura chalega chahe element mil gaya ho.



Key Takeaways

TAKEAWAY

- **Array Definition:** Same type ke elements ka collection jo continuous memory mein store hote hain.
- **Declaration:** `int arr[5];` — 5 integers ka array.
- **0-Based Indexing:** Array hamesha 0 se start hota hai. Last index = size - 1.
- **Initialization:** `int arr[] = {1, 2, 3};` — compiler size khud le leta hai.
- **Memory Layout:** Continuous addresses. Har element `sizeof(data_type)` bytes door.
- **Address Formula:** Base Address + (index × size)
- **Traversal:** `for(int i = 0; i < n; i++)` — sabse common pattern.
- **Input/Output:** Loop se `cin >> arr[i]` aur `cout << arr[i]`
- **Highest Element:** Pehle element ko highest maan lo, phir compare karte jao.
- **Linear Search:** Har element check karo, milne pe return kar do.

Concept	Syntax / Formula	Example
Declaration	<code>int arr[5];</code>	5 integers ka array
Initialization	<code>int arr[] = {1,2,3};</code>	Auto size = 3
Access	<code>arr[index]</code>	<code>arr[0] = 70</code>
First Index	0	<code>arr[0]</code> = first element
Last Index	$n - 1$	<code>arr[4]</code> for size 5
Address	$\text{Base} + (i \times \text{size})$	$1000 + (2 \times 4) = 1008$
Traverse	<code>for(i=0; i<n; i++)</code>	All elements access
Linear Search	Compare each element	$O(n)$ complexity

INTERVIEW TIP

Interview mein sabse common question: **"Array ka size runtime pe kyun nahi badh sakta?"** Answer: Kyunki array **stack memory** mein allocate hota hai aur uski size **compile time** pe fix ho jati hai. Isliye dynamic arrays (Vectors) aate hain — wo baad mein padhenge.

Key Insight: Array programming ka **foundation** hai. Bina array ke data structures jaise *Strings, 2D Arrays, Vectors, Stacks, Queues* — kuch bhi samajhna mushkil hai. Array ke concepts ko **strong karo**, baaki sab aasan lagega.